Nghiên cứu đề tài: Phòng trưng bày 3D

Nov 3, 2026 · @Someone

Tổng quan đề tài

Đề tài xây dựng một **phòng trưng bày 3D**: các vật thể (bình gốm, ly, chai, quả cầu, xuyến) được chương trình sinh ra từ đường cong Bezier và mặt tròn xoay, rồi được hiển thị với camera, chiếu sáng và khử mặt khuất do nhóm tự cài đặt. Người xem đi lại trong phòng và bật/tắt từng thuật toán để so sánh.

Mục tiêu: một chương trình C++/OpenGL chạy được trên macOS và Windows, phủ các chương 4–6 của môn, và mỗi thuật toán có một màn so sánh dùng được khi thuyết trình.

Phạm vi: đây là dự án đồ họa, không phải game. Không làm vật lý, âm thanh hay AI. Mô hình 3D chủ yếu do chương trình tự sinh; nếu có nạp file `.obj` thì chỉ là phần cộng thêm.

Chức năng	Kiến thức trong môn	Lab tương ứng
Vẽ đường cong Bezier bằng chuột, sinh mặt tròn xoay	CG.5 Curves and Surfaces	Lab 05
Biến đổi, đặt vật vào phòng	CG.4 Biến đổi 3D	Lab 04 (đã làm)
Camera, chiếu song song/phối cảnh, phóng to	CG.4 Quan sát 3D	Lab 04 (đã làm)
Back-face, Z-buffer, Painter	CG.6 Khử mặt khuất	Lab 06
Lambert (Flat), Gouraud, Phong	CG.6 Tô bóng	Lab 07
Vẽ đường cong 2D trên màn hình soạn thảo	CG.3 Đồ họa 2D	Lab 01

Code `Lab_BienDoi3D` hiện có đã có sẵn ma trận biến đổi 4×4, `PolygonMesh` (danh sách đỉnh và mặt), pháp tuyến Newell và hệ quan sát. Nhóm có thể dùng chúng làm lõi, không phải viết lại từ đầu.

Mô hình hóa: từ đường cong đến lưới tam giác

Mỗi vật trưng bày là một **đường cong biên dạng (profile)** trong mặt phẳng xy, quay quanh trục y để thành mặt, rồi được lấy mẫu thành lưới tam giác có pháp tuyến. Đây là cách các môn đồ họa đại học (ví dụ UW CSE 457) dạy phần mặt tròn xoay.

Đường cong Bezier bậc 3

Bốn điểm điều khiển P0..P3, tham số t từ 0 đến 1:

```
B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3
```

- **de Casteljau:** nội suy tuyến tính lặp lại giữa các điểm điều khiển. Ổn định số và vẽ được các đoạn trung gian, nên dùng để minh họa trực quan khi thuyết trình.
- **Nối nhiều đoạn:** một chiếc bình cần 2–4 đoạn Bezier. Giữ liên tục C1 bằng cách cho P2 của đoạn trước, điểm nối và P1 của đoạn sau thẳng hàng.
- **Phương án thay thế:** B-spline bậc 3 đều tự đảm bảo C2 và kéo một điểm chỉ ảnh hưởng cục bộ. Nếu còn thời gian, cho người dùng chuyển qua lại Bezier/B-spline là một điểm cộng.

Sinh mặt tròn xoay

Quay profile C(v) = (Cx(v), Cy(v), 0) một góc θ = 2πu quanh trục y:

```
S(u,v) = \big( C_x(v)\cos 2\pi u,\; C_y(v),\; -C_x(v)\sin 2\pi u \big)
```

1. Lấy M mẫu trên profile (mỗi mẫu là một **vòng**) và N+1 góc θi = 2πi/N (mỗi góc là một **lát**). Lát đầu và lát cuối trùng nhau để khép mặt.
2. Đỉnh P[i][j] = Ry(θi) · C[j]. Dùng lại ma trận quay có sẵn trong `transforms_3d.hpp`.
3. Mỗi ô lưới chia hai tam giác: (P[i][j], P[i+1][j], P[i+1][j+1]) và (P[i][j], P[i+1][j+1], P[i][j+1]). Giữ cùng chiều đỉnh để pháp tuyến hướng ra ngoài, khớp với quy ước "ngược chiều kim đồng hồ" trong `polygon_mesh.hpp`.
4. Giá trị gợi ý: M = 32–64, N = 48–72. Một bình khoảng 3.000–9.000 tam giác, đủ mượt cho Gouraud.

Pháp tuyến theo đỉnh

Pháp tuyến theo đỉnh là điều kiện để Gouraud và Phong khác Flat. Có hai cách:

- **Giải tích (nên dùng):** tiếp tuyến profile T = (Cx′, Cy′, 0), quay 90° thành n = (Cy′, −Cx′, 0), chuẩn hóa, rồi quay theo θi giống đỉnh. Đạo hàm Bezier có công thức đóng: B′(t) = 3[(1−t)²(P1−P0) + 2t(1−t)(P2−P1) + t²(P3−P2)].
- **Trung bình mặt:** cộng pháp tuyến Newell của các mặt kề rồi chuẩn hóa. Dùng cho mặt không tham số như file `.obj`.
- **Bẫy cần tránh:** tại điểm trên trục (Cx = 0, đáy hoặc miệng bình), các tam giác bị suy biến. Gom các đỉnh trên trục thành một đỉnh và đặt pháp tuyến (0, ±1, 0).

Các mặt có sẵn (cầu, trụ, nón, xuyến) cũng là mặt tròn xoay với profile là nửa đường tròn, đoạn thẳng hoặc đường tròn lệch tâm. Vì vậy cùng một hàm sinh lưới phục vụ được toàn bộ Lab 05.

Cảnh và camera

Camera cần hai chế độ: **đi bộ trong phòng** (góc nhìn thứ nhất) và **xoay quanh một vật** (orbit). Cả hai đều quy về một ma trận look-at từ ba đại lượng: vị trí mắt E, điểm nhìn C và vectơ hướng lên Up.

Ma trận quan sát (look-at)

```
f = \frac{C - E}{\lVert C - E \rVert},\quad s = \frac{f \times Up}{\lVert f \times Up \rVert},\quad u = s \times f
```

Ma trận view có ba hàng s, u, −f và cột tịnh tiến (−s·E, −u·E, f·E). Đây chính là phép biến đổi hệ quan sát trong slide CG.4; tự cài nó thay vì gọi `gluLookAt` giúp nhóm giải thích được khi bị hỏi.

Hai kiểu camera

Chế độ	Trạng thái lưu	Điều khiển	Tính E và C
Đi bộ (FPS)	vị trí E, góc yaw, góc pitch	W/A/S/D di chuyển, kéo chuột xoay	f = (cos pitch·sin yaw, sin pitch, −cos pitch·cos yaw), C = E + f
Xoay quanh vật (orbit)	tâm vật C, bán kính R, góc θ, góc φ	kéo chuột xoay, cuộn để phóng to	E = C + R·(cos φ·sin θ, sin φ, cos φ·cos θ)

Giới hạn pitch và φ trong khoảng ±89° để f không song song với Up (nếu không, tích có hướng băng 0 và hình bị lật). Trong chế độ đi bộ, kẹp E trong hộp của căn phòng để không đi xuyên tường. Đây là "va chạm" duy nhất cần làm.

Phép chiếu

- **Phối cảnh:** góc nhìn dọc (fovy) 45–60°, tỉ lệ khung hình lấy từ kích thước cửa sổ, near = 0.1, far = 100. Tỉ lệ far/near quá lớn sẽ làm Z-buffer mất độ chính xác (hiện tượng nhấp nháy z-fighting).
- **Song song (trực giao):** hộp nhìn đối xứng, kích thước tỉ lệ với bán kính orbit, để khi chuyển kiểu chiếu vật không đột ngột to hoặc nhỏ.

- Bấm một phím (ví dụ `P`) để chuyển hai kiểu chiếu. `viewer3d.cpp` đã làm được việc này cho Lab 04.

Chọn vật bằng chuột

- **Color picking (dễ nhất):** vẽ lại cảnh vào back buffer, mỗi vật một màu ID, tắt chiếu sáng, rồi đọc pixel dưới con trỏ bằng `glReadPixels`. Khoảng 30 dòng code.
- **Băn tia (ray casting):** đổi tọa độ chuột ngược qua ma trận chiếu và view thành một tia, rồi kiểm tra giao với hộp bao của từng vật. Cách này thể hiện hiểu biết về ma trận nghịch đảo, nhưng mất nhiều công hơn.

Khi đã chọn được vật, camera chuyển sang orbit quanh vật đó và hiện bảng thông tin (tên, số tam giác, vật liệu). Đây là khoảnh khắc "wow" tốt cho phần demo.

Chiếu sáng và tô bóng

Cả ba kiểu tô bóng dùng chung một **mô hình chiếu sáng Phong**; chúng chỉ khác nhau *ở chỗ tính* công thức: mỗi mặt một lần, mỗi đỉnh một lần, hay mỗi pixel một lần. Nên cài công thức một lần rồi gọi nó ở ba chỗ khác nhau.

Mô hình Phong

```
I = k_a I_a + \sum_{\text{đèn}} \Big[ k_d \max(N\cdot L, 0) + k_s \max(R\cdot V, 0)^{n} \Big] I_L
```

- N: pháp tuyến đơn vị; L: hướng tới đèn; V: hướng tới mắt; R = 2(N·L)N − L là tia phản xạ.
- ka, kd, ks, n là thông số vật liệu; n càng lớn thì vệt sáng càng nhỏ và gắt.
- **Blinn-Phong:** thay R·V bằng N·H với H = (L + V)/|L + V|. Nhanh hơn, và không bị cắt vệt sáng khi góc giữa V và R lớn hơn 90°. Nên có phím chuyển Phong/Blinn-Phong để làm thêm một màn so sánh.

Ba kiểu tô bóng

Kiểu	Tính ánh sáng ở đâu	Pháp tuyến dùng	Trông như thế nào trên bình gốm
Flat (Lambert/hăng)	mỗi mặt một lần	pháp tuyến mặt	thấy rõ từng mặt, như được cắt gọt
Gouraud	tại đỉnh, nội suy **màu**	pháp tuyến đỉnh	mượt, nhưng vệt sáng bị nhòe, có hình ngôi sao, có thể mất hẳn khi lưới thưa

Kiểu	Tính ánh sáng ở đâu	Pháp tuyến dùng	Trông như thế nào trên bình gốm
Phong	tại pixel, nội suy **pháp tuyến**	pháp tuyến đỉnh, chuẩn hóa lại ở pixel	vệt sáng tròn, sắc nét ngay cả trên lưới thưa

Màn so sánh hiệu quả nhất: giảm số lát N xuống khoảng 12. Khi đó Gouraud làm vệt sáng nhấp nháy hoặc biến mất khi xoay, còn Phong vẫn giữ được. Điều này chứng minh trực quan vì sao Phong tốn chi phí hơn nhưng đẹp hơn.

Cài đặt

- **Không dùng** `glEnable(GL_LIGHTING)` cho phần chấm điểm: nó chỉ có Flat và Gouraud, và công thức bị ẩn trong driver. Có thể giữ nó làm "đáp án" để đối chiếu kết quả.
- **Cách đề xuất:** viết hai shader GLSL 1.20 nhỏ. Gouraud tính Phong trong vertex shader, Phong tính trong fragment shader, Flat tính trên CPU rồi gửi màu mặt. Mỗi shader khoảng 30 dòng, công thức hiện rõ trong code.
- **Cách thuần CPU** (nếu giảng viên muốn thấy thuật toán tô từng pixel): tự rasterize tam giác bằng tọa độ trọng tâm, nội suy màu hoặc pháp tuyến, ghi vào mảng pixel rồi hiển thị bằng `glDrawPixels`. Xem mục Khử mặt khuất: cùng một bộ rasterizer phục vụ cả Z-buffer.

Vật liệu gợi ý

Vật liệu	kd (màu nền)	ks	n
Gốm tráng men	màu men, sáng	0.5	64
Kim loại (đồng, vàng)	màu kim loại, tối	0.8, cùng màu kim loại	128
Nhựa	màu tươi	0.3, màu trắng	32
Đá, đất nung	màu đất	0.05	8

Các số trên là điểm xuất phát để chỉnh bằng mắt, không phải giá trị chuẩn. Thêm phím đổi vật liệu ngay khi demo sẽ cho thấy ảnh hưởng của ks và n.

Khử mặt khuất

Ba thuật toán của Lab 06 nên chạy trên **cùng một cảnh** và chuyển bằng phím. Mỗi thuật toán có một trường hợp sai riêng, và cảnh trong phòng trưng bày nên cố ý chứa trường hợp đó để chứng minh khi thuyết trình.

Thuật toán	Ý tưởng	Chi phí	Khi nào sai	Cách chứng minh khi demo
Back-face culling	bỏ mặt có N·(P − E) > 0, tức mặt quay lưng về mắt	mỗi mặt một tích vô hướng; bỏ khoảng một nửa số mặt	vật lõm, hoặc nhiều vật chồng nhau: mặt quay ra vẫn có thể bị che	bật riêng Back-face với chiếc bình miệng loe: thành trong và ngoài vẽ sai thứ tự; hiển thị số tam giác bị bỏ
Painter (depth-sort)	săp xếp mặt theo độ sâu từ xa đến gần rồi vẽ đè	săp xếp O(n log n) mỗi khung hình	hai mặt cắt nhau, hoặc ba mặt chồng vòng tròn (không có thứ tự đúng)	đặt một quả cầu căm xuyên qua một tấm phẳng: Painter vẽ sai, Z-buffer vẽ đúng
Z-buffer (depth-buffer)	mỗi pixel lưu độ sâu gần nhất; chỉ ghi nếu pixel mới gần hơn	thêm một mảng độ sâu (800×600 số float là khoảng 1,9 MB)	chỉ sai khi độ chính xác không đủ (z-fighting); không xử lý được vật trong suốt	hiển thị chính mảng độ sâu dưới dạng ảnh xám: gần thì sáng, xa thì tối

Hai cách cài Z-buffer

- **Dùng GPU:** `glEnable(GL_DEPTH_TEST)`. Chỉ một dòng, nhưng không thể hiện được thuật toán.
- **Tự cài trên CPU (nên làm):** một bộ rasterizer khoảng 150–250 dòng. Chiếu ba đỉnh ra màn hình, duyệt pixel trong hộp bao của tam giác, dùng tọa độ trọng tâm để kiểm tra pixel nằm trong và để nội suy z, so với mảng độ sâu, rồi ghi màu. Đây cũng là nền để làm Phong trên CPU. Loạt bài tinyrenderer và Scratchapixel hướng dẫn đúng phần này.
- **Lưu ý khi nội suy:** trong phép chiếu phối cảnh, độ sâu thay đổi tuyến tính theo 1/z trên màn hình, không phải theo z. Nội suy 1/z (và chia lại khi nội suy màu, pháp tuyến hay tọa độ texture) để tránh hình bị méo (Scratchapixel).

Thứ tự trong pipeline: Back-face chạy trước để bớt số mặt, sau đó đến Painter hoặc Z-buffer. Bật Back-face cùng Z-buffer cho kết quả đúng và nhanh nhất; các tổ hợp còn lại dùng để minh họa lỗi.

Tính năng cộng điểm

Chỉ làm phần này sau khi các chức năng bắt buộc đã chạy ổn. Bảng dưới xếp theo tỉ lệ "đẹp thêm / công sức", việc đáng làm nhất ở trên cùng.

Tính năng	Cách làm	Công sức ước tính	Người phụ trách
Texture (vân gỗ cho sàn, vân đá cho bục)	đọc ảnh PNG/JPG bằng thư viện một file stb_image.h; mặt tròn xoay có sẵn tọa độ texture chính là (u, v) dùng khi sinh lưới	1–2 ngày	B
Nhiều nguồn sáng di chuyển được	cộng tổng trong công thức Phong (đã có dạng tổng); vẽ mỗi đèn thành một quả cầu nhỏ, kéo được bằng chuột	1 ngày	C
Bóng đổ phẳng xuống sàn	chiếu từng vật lên mặt phẳng sàn theo tâm là vị trí đèn (một ma trận 4×4 chiếu phối cảnh), vẽ màu tối bán trong suốt	1 ngày	C
Lưu/tải vật thể	ghi các điểm điều khiển Bezier ra file văn bản, hoặc xuất lưới ra `.obj` để mở được bằng Blender	nửa ngày	A
Nạp file `.obj` từ ngoài	đọc các dòng `v`, `vn`, `vt`, `f`; thử với ấm trà Utah hoặc thỏ Stanford	1 ngày	B
Menu chuột phải và dòng chữ trạng thái	`glutCreateMenu`, `glutBitmapCharacter` để hiện chế độ đang bật và FPS	nửa ngày	cả nhóm

Ma trận bóng đổ lên mặt phẳng y = 0 từ đèn điểm L = (lx, ly, lz) chiếu điểm P thành:

```
P' = \left( \frac{l_y p_x - l_x p_y}{l_y - p_y},\; 0,\; \frac{l_y p_z - l_z p_y}{l_y - p_y} \right)
```

Nâng sàn lên một chút (khoảng 0.001) hoặc dùng `glPolygonOffset` để bóng không nhấp nháy với sàn.

Không nên làm trong đồ án này: shadow mapping, phản xạ thật, vật lý. Công sức cao, dễ lỗi, và nằm ngoài nội dung môn.

Công nghệ và môi trường

Giữ nguyên bộ công cụ đang dùng trong các lab: **C++17, OpenGL cũ (compatibility) và GLUT, build bằng Makefile**. Chỉ bổ sung shader GLSL 1.20 và một thư viện đọc ảnh. Đây là cách ít rủi ro nhất để cả ba máy của nhóm đều chạy được.

Thành phần	Lựa chọn	Lý do
Ngôn ngữ	C++17	đã dùng trong `Lab_BienDoi3D`; có `std::vector`, `std::array` cho ma trận
Cửa sổ, bàn phím, chuột	GLUT (`-framework GLUT` trên macOS), freeglut trên Windows (bản cho MinGW)	giống API các lab và slide CG.2
Đồ họa	OpenGL 2.1 compatibility	GLUT của Apple tạo context legacy 2.1; vẫn dùng được `glBegin` / `glEnd` và shader GLSL 1.20
Shader	GLSL `#version 120`	chạy được trên context 2.1 của macOS và trên mọi driver Windows
Đọc ảnh texture	stb_image.h	một file header, không phải cài đặt
Ma trận, vectơ	tự viết (đã có `Matrix4`, `Point3`)	giảng viên thấy được phép biến đổi; không phụ thuộc GLM
Quản lý code	Git và một repo GitHub chung	ba người code song song, mỗi người một nhánh

Lưu ý về macOS

- Apple đã ngừng phát triển (deprecated) OpenGL từ macOS 10.14 Mojave năm 2018 và khuyên dùng Metal; ứng dụng OpenGL vẫn chạy nhưng trình biên dịch báo cảnh báo (AppleInsider). Thêm `-DGL_SILENCE_DEPRECATION` vào `CXXFLAGS` để tắt các cảnh báo này.
- Muốn dùng OpenGL 4.1 (cao nhất trên macOS) thì phải xin core profile và bỏ toàn bộ hàm cũ (Castle Engine). Không đáng với đồ án này: mọi thứ cần làm đều có trong 2.1.
- Makefile hiện tại đã tự chọn cờ link theo hệ điều hành (`-framework OpenGL -framework GLUT` trên macOS, `-lglut -lGL` trên Linux). Cần thêm nhánh cho Windows/MinGW: `-lfreeglut -lopengl32 -lglu32`.

Cấu trúc thư mục đề xuất

```
phong-trung-bay/
  include/   math3d.hpp  mesh.hpp  camera.hpp  lighting.hpp  raster.hpp
  src/
    modeling/    bezier.cpp  revolve.cpp  editor2d.cpp      (A)
    scene/       room.cpp  camera.cpp  picking.cpp          (B)
    render/      shading.cpp  zbuffer.cpp  painter.cpp      (C)
    main.cpp     ghep 3 phan, bang phim tat
  shaders/   gouraud.vert/.frag  phong.vert/.frag
  assets/    textures/  models/  (.obj, .txt diem dieu khien)
  Makefile  README.md
```

`mesh.hpp` (struct chứa đỉnh, pháp tuyến, tọa độ texture, tam giác) là hợp đồng giữa ba người: A tạo ra `Mesh`, B đặt nó vào cảnh, C vẽ nó. Cần thống nhất file này ngay tuần đầu.

Tài liệu và dự án tham khảo

Nên đọc theo thứ tự của người phụ trách: mỗi thành viên chỉ cần đọc kỹ 2–3 nguồn của phần mình.

Nguồn	Nội dung dùng cho đề tài	Dành cho
Slide môn học CG.4, CG.5, CG.6, CG.7 (thư mục `Slides VI`)	công thức và ký hiệu giảng viên dùng; phải bám theo khi viết báo cáo	cả nhóm
Hearn, Baker & Carithers, *Computer Graphics with OpenGL*, bản 4, 2014	giáo trình chính của môn: Bezier, B-spline, quan sát 3D, khử mặt khuất, tô bóng	cả nhóm
Angel & Shreiner, *Interactive Computer Graphics*, bản 6, 2012	cài đặt chiếu sáng bằng shader	C
UW CSE 457: Surfaces of Revolution	sinh lưới vòng × lát, chia tam giác, pháp tuyến giải tích	A
LearnOpenGL: Basic Lighting và Materials	mô hình Phong trong shader, so sánh Gouraud và Phong, bảng vật liệu. Bài dùng GLSL 3.30 nên phải chuyển sang 1.20	C
tinyrenderer	khóa ngắn 14 bài tự viết renderer bằng C++ (khoảng 500 dòng): rasterize tam giác, tọa độ trọng tâm, Z-buffer, camera, tô bóng, texture	C, B

Nguồn	Nội dung dùng cho đề tài	Dành cho
Scratchapixel: Rasterization	Z-buffer và nội suy đúng phối cảnh, giải thích kỹ toán học	C
stb (`stb_image.h`)	đọc JPG, PNG, BMP, TGA cho texture	B
freeglut cho MinGW	build GLUT trên Windows cho thành viên không dùng Mac	B
Code `Lab_BienDoi3D` của nhóm	`Matrix4`, `PolygonMesh`, pháp tuyến Newell, `viewer3d.cpp`	cả nhóm

Mô hình mẫu để thử (ấm trà Utah, thỏ Stanford) có sẵn dạng `.obj` trên mạng. Nếu dùng, ghi rõ nguồn trong báo cáo và phân biệt với các vật do nhóm tự sinh.

Phân công, kế hoạch và rủi ro

Mỗi người có một mảng độc lập; ba người chỉ gặp nhau ở struct `Mesh` và hàm `main`. Mốc quan trọng nhất là cuối tuần 3: khi đó phải có đủ tính năng bắt buộc, để nếu trễ thì chỉ mất phần cộng điểm.



kế hoạch 5 tuần · 3 người · 4 mốc

Tuần 4 cả nhóm ghép code và làm tính năng cộng điểm; sau mốc "đóng băng" chỉ sửa lỗi, không thêm tính năng.

Rủi ro và cách phòng

Rủi ro	Cách phòng
Tuần 4 ghép code bị vỡ vì mỗi người hiểu `Mesh` một kiểu	chốt `mesh.hpp` trong tuần 1; cuối mỗi tuần ghép thử trên nhánh `main`
Máy Windows của một thành viên không build được	Makefile đa nền tảng xong trong tuần 1, cả ba người build thử chương trình "hello"
Bộ rasterizer trên CPU chạy chậm	chỉ dùng CPU trong màn so sánh, cửa sổ 640×480; chế độ đi bộ bình thường dùng GPU
Shader báo lỗi trên macOS	viết đúng `#version 120`; luôn in `glGetShaderInfoLog` khi biên dịch
Một thành viên bị chậm	phần bắt buộc của mỗi người chạy độc lập; tính năng cộng điểm bỏ được mà không ảnh hưởng phần khác
Giảng viên hỏi sâu về công thức	ai làm phần nào thuyết trình phần đó; ghi công thức và số trang slide vào comment trong code
Máy chiếu hoặc máy demo hỏng hôm thuyết trình	quay sẵn video demo 2–3 phút trong tuần 5

Câu hỏi còn mở

- [ ] Hạn nộp và ngày thuyết trình là khi nào?
- [ ] Giảng viên có bắt buộc tự cài thuật toán trên CPU, hay cho dùng shader?
- [ ] Báo cáo có mẫu riêng không (số trang, định dạng)?
- [ ] Thành viên nào nhận phần A, B, C?